electronics

MDPI

Review

Systematic Mapping Study of Test Generation for Microservices: Approaches, Challenges, and Impact on System Quality

Tingshuo Miao *, Asif Imtiaz Shaafi and Eunjee Song *

Department of Computer Science, Baylor University, Waco, TX 76798, USA
* Correspondence: tingshuo_miao2@baylor.edu (T.M.); eunjee_song@baylor.edu (E.S.)

Abstract: Software development underwent a significant shift with the adoption of microservice architecture. Such architecture offers modularity and scalability but introduces new testing challenges. This review aims to identify, categorize, and analyze key testing methods used in microservices-based systems, emphasizing how these methods address architectural challenges and affect overall system quality. The distributed nature and asynchronous communication patterns of microservices architecture create a pressing need for robust and adaptive testing approaches. We systematically reviewed relevant papers, categorized approaches, and analyzed their strengths, limitations, and impact on system reliability. Our findings reveal that contract testing, behavior-driven development, and automated regression testing are widely practiced, yet there is a notable shortage of dedicated test generation tools designed specifically for microservices (i.e., the automated creation of test cases and test scripts tailored to verify microservices' functionality and interactions). Additionally, this study underscores the necessity of specialized tools and advocates mock services as a solution for stabilizing complex inter-service dependencies. This study offers insights that advance both research and practice, ultimately improving microservices system reliability.

Keywords: microservices testing; test generation; automated testing; systematic mapping study; quality assurance; contract testing

check for updates

Academic Editor: Manuel Mazzara

Received: 21 January 2025
Revised: 25 March 2025
Accepted: 27 March 2025
Published: 30 March 2025

Citation: Miao, T.; Shaafi, A.S.; Song, E. Systematic Mapping Study of Test Generation for Microservices: Approaches, Challenges, and Impact on System Quality. *Electronics* **2025**, *14*, 1397. https://doi.org/10.3390/electronics14071397

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Software development has shifted toward microservice architecture because it enables modular, independently deployable services and flexible scaling [1–3]. This shift addresses the limitations of monolithic systems, which couple all functionalities into one large deployment [4]. While microservices improve agility and scalability, they introduce intricate testing challenges. Each service releases its own updates, and asynchronous communication, evolving versions, and distributed dependencies complicate test design. When systems rely on independent services that must collaborate seamlessly, conventional testing methods often become insufficient or cumbersome.

Researchers have explored core aspects of microservices, including performance [5] and testing strategies [6], and these papers offer valuable insights into various methodologies and tools. Some prioritize performance benchmarks and fault injection, whereas others propose contract-based validations and mock services to isolate each component. However, existing efforts do not comprehensively address automated test generation needs, especially when services rapidly evolve. Traditional workflows struggle to maintain adequate coverage for large, continuously evolving microservice deployments without incurring excessive maintenance overhead.

This study conducts a systematic mapping study (2015–2024) and identifies 47 relevant papers. Our goal is to examine which test generation practices have gained traction in recent microservices research, as well as the strengths and limitations of these practices in dynamic, distributed environments. We focus on critical areas such as test isolation, contract conformance, resilience validation, and integration with CI/CD pipelines.

This paper consolidates common testing approaches, highlights key research gaps, and proposes potential paths to refine microservices testing. We show that coverage-oriented strategies, combined with contract and fault-based testing, can capture real-world behaviors more thoroughly. We also present best practices and highlight current challenges in automated test generation and ongoing maintenance efforts.

In the remainder of this paper, Section 2 explains our systematic mapping study process, Section 3 presents the key findings, Section 4 addresses prevalent test generation approaches and challenges, and Section 5 discusses impacts on quality, followed by a discussion in Section 6. Threats to the validity of the study are given in Section 7. Finally, Section 8 concludes the study with future directions.

2. Research Method

We conducted a systematic mapping study (SMS) by following the guidelines provided by Petersen et al. [7], supplemented by the filtering principles from Ahmed et al. [8]. These guidelines offered a structured framework for identifying, classifying, and analyzing the software engineering literature. We explicitly followed core SMS steps, including defining research questions, specifying a search strategy, screening, and data extraction. Additionally, we refined our methodology to account for the distributed nature and frequent updates inherent in microservices. For example, we prioritized papers addressing test generation in microservices, comparisons with monolithic systems, and issues related to resilience and asynchronous communication. These adaptations ensured our analysis effectively captured complexities unique to microservices testing.

Our research method consisted of three high-level phases—*Search*, *Filter*, and *Map*—adapted from the guidelines, as illustrated in Figure 1. We derived these phases from the systematic mapping process outlined by Petersen et al. [7], consolidating them to align with the context of microservices test generation.

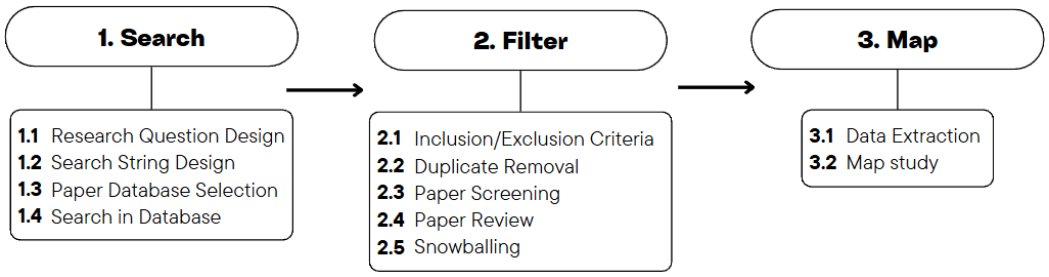


Figure 1. Phases of the systematic mapping study (SMS).

2.1. Search

During the *Search* phase, we defined research questions, selected databases, and constructed a search string to locate papers related to microservices test generation. We chose this sequence of activities based on the adapted guidelines in [7] to ensure that the search strategy adequately reflects the scope of our study.

2.1.1. Research Questions

RQ1: *What are the key approaches and challenges in test generation for microservices?*

The primary aim of this study is to identify and analyze the most widely adopted testing methods for microservice architectures and the challenges that testers and developers face during the test generation process. We also aim to examine techniques for thorough testing of individual services to ensure high reliability while identifying major obstacles that practitioners encounter. This approach provides suggestions that foster innovation and advance current microservices testing practices.

To achieve this goal, we subdivided RQ1 into several specific questions related to testing techniques, differences from traditional monolithic testing, available tools and frameworks, and the relative strengths and weaknesses of each method. Table 1 presents these questions and outlines their respective motivations.

Table 1. Specific research questions and their motivations for RQ1.

ID	Specific Research Question	Main Motivation
RQ1.1	What are the most prevalent testing techniques for microservices?	Identifying and categorizing the most commonly used testing techniques helps practitioners select the most suitable methods. Due to the distributed nature of microservices, diverse testing strategies are essential.
RQ1.2	How do these testing techniques compare with those applied to traditional monolithic application testing?	Understanding how microservices testing differs from monolithic testing highlights unique challenges and advantages, guiding practitioners in adopting appropriate strategies.
RQ1.3	What tools and frameworks are available specifically for test generation in microservices?	This question aims to identify dedicated tools and frameworks that facilitate automated test generation for microservices, assessing their effectiveness and limitations.
RQ1.4	What are the strengths and weaknesses of various test methods in the context of microservices?	Evaluating the advantages and drawbacks of different testing techniques helps determine which methods are more effective, scalable, or challenging in microservices environments.

RQ2: *What is the impact of test generation on the quality of microservices-based systems?*

We aim to understand how test generation influences the quality of microservice-based applications. Our goal is to identify which types of generated tests are most useful for increasing robustness, fault tolerance, and performance in microservices. We aim to determine the appropriate strategies for testing such applications by assessing the results of various testing approaches used. Our research informs practitioners about the relationship between testing coverage breadth and efficiencies gained in service delivery, providing valuable insights for improving their processes. Table 2 summarizes the primary motivation behind each specific research question under RQ2.

Table 2. Specific research questions and motivations for RQ2.

ID	Specific Research Question	Main Motivation
RQ2.1	What factors influence the testing of microservices?	Understanding the key factors that impact microservices testing, such as service interactions and deployment environments, helps improve test strategies and reliability.
RQ2.2	Which metrics are crucial for evaluating the quality of microservice tests or test cases?	Identifying effective test quality metrics ensures that testing strategies are properly assessed and optimized for robustness and reliability.
RQ2.3	How does effective test generation enhance the quality of microservices?	Investigating the role of test generation in improving software quality provides insights into best practices for increasing fault tolerance and stability.
RQ2.4	What is the relationship between test coverage and test generation efficiency in microservices?	Analyzing how test coverage impacts test generation efficiency helps balance thorough testing with practical resource constraints.

2.1.2. Paper Database Selection

Selecting appropriate databases is crucial for ensuring comprehensive coverage of relevant papers in a systematic mapping study. We evaluated five indexing services—IEEE Xplore, ACM Digital Library, SpringerLink, Scopus, and Web of Science—based on suggestions from a guidelines paper by Petersen et al. [7] and another well-cited systematic review paper by Dyba et al. [9]. These databases provide extensive coverage of research in software engineering and microservices testing. Each database indexes unique conferences and journals, although Web of Science and Scopus share significant overlap [10]. We included all five databases to ensure that no significant contributions were missed.

2.1.3. Search in the Database

The search strings for this study were constructed based on the Population, Intervention, Comparison, and Outcomes (PICO) criteria, as guided by Petersen et al. [7]. The *Population* focuses on microservices architectures, the *Intervention* targets test generation methods, *Comparison* involves existing testing approaches, and *Outcomes* refer to the quality and effectiveness of tests.

To construct our final search string, we conducted multiple rounds of pilot queries using the IEEE Xplore and ACM Digital Library databases. For this incremental query construction and validation process, we first categorized keywords into three groups: (1) primary terms directly related to microservices, (2) secondary terms concerning test generation and automation, and (3) application terms reflecting approaches, challenges, and impacts. Multiple iterations were performed to refine the search string, aiming to maximize the coverage of relevant papers while minimizing irrelevant results.

The effectiveness of each search string was assessed by running preliminary searches with a pilot set of 10 carefully selected relevant papers. These pilot papers were essential for verifying the comprehensiveness of the search strings. The evaluation results are summarized in Table 3, which illustrates how our final search string was constructed through this iterative validation process.

Table 3. Results of pilot queries in IEEE Xplore and ACM Digital Library.

Set #	Search Strings	# Results	# Missing Papers
Set #1	(“test generation“) AND (“microservices”)	119	9
Set #2	(“test generation” OR “test automation”) AND (“microservices” OR “microservice architecture”)	191	8
Set #3	(“test” OR “test generation” OR “test automation”) AND (“microservices” OR “microservice architecture”)	2 992	1
Set #4	(“test” OR “test generation” OR “test automation”) AND (“microservices” OR “microservice architecture”) AND (“approaches” OR “techniques” OR “challenges” OR “problems”)	2 035	3
Set #5	(“test” OR “test generation” OR “test automation”) AND (“microservice” OR “microservice architecture”) AND (“approaches” OR “techniques” OR “methods” OR “challenges” OR “problems” OR “impact” OR “effect” OR “quality”)	2 096	0

As shown in Table 3, the first four preliminary search strings returned numerous irrelevant papers or missed several relevant pilot papers. The final search string (Set #5) was selected, as it retrieved all relevant pilot papers, precisely aligned with the scope, and addressed the research questions effectively. Applying the final search string (Set #5) across all five selected databases resulted in 2339 papers, as shown in Table 4. This set of papers

served as the starting point of our *Filter* phase, a multi-step selection process designed to ensure both relevance and quality.

Table 4. Number of papers retrieved.

Database	Number of Papers Retrieved
SpringerLink	224
ACM Digital Library	1753
IEEEXplore	343
Scopus	14
Web of Science	5
Total	2339

2.2. Filter

Our selection methodology was guided by the principles outlined in the paper by Ahmed et al. [8] with adaptations to align it with the systematic mapping study guidelines [7] while maintaining the focus on microservices testing. In this phase, we selected the most relevant and valuable papers using well-defined filtering criteria for the initial set of publications. More details on the selection criteria and process are covered in the following two subsections.

2.2.1. Selection Criteria

The initial selection of papers was based on the title and abstract, which were reviewed to determine the relevance to our research questions. The full list of adopted inclusion and exclusion criteria is presented in Tables 5 and 6, respectively.

Table 5. Inclusion criteria.

ID	Criteria
I1	Papers discussing approaches and challenges in test generation for microservices.
I2	Papers comparing testing techniques for microservices with traditional monolithic applications.
I3	Papers presenting tools and frameworks specifically for test generation in microservices.
I4	Empirical studies evaluating the quality of microservices-based systems through test generation.
I5	Papers published in peer-reviewed journals or conferences.

Table 6. Exclusion criteria.

ID	Criteria
E1	Papers not directly related to microservice testing.
E2	Papers not available in full text.
E3	Papers not written in English.
E4	Books and non-peer-reviewed sources.

2.2.2. Selection Process

We performed four main tasks to refine the 2339 initially retrieved papers (see Table 4) into our final set of 48 (see Table 7). Figure 2 illustrates the selection process via an adapted PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) flow diagram [11].

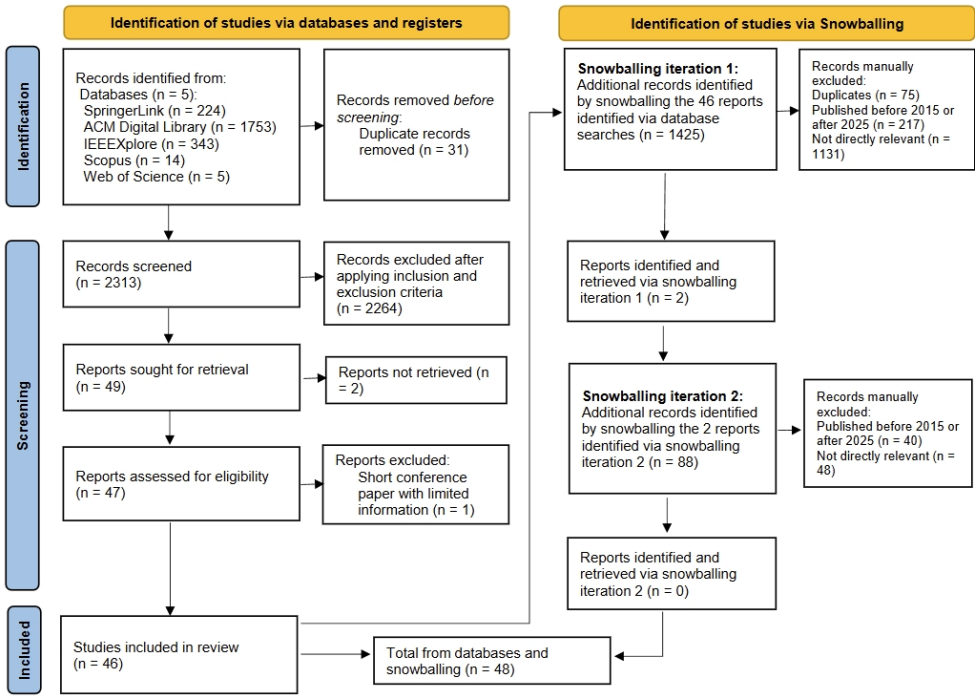


Figure 2. Adapted PRISMA flow diagram for the selection process.

1. **Removing Duplicates:** We identified duplicate papers resulting from indexing overlaps across databases. A total of 31 duplicates were removed to ensure uniqueness in our dataset.
2. **Title and Abstract Screening:** Next, we manually reviewed the titles and abstracts of the 2313 remaining papers. Each paper was evaluated against predefined inclusion and exclusion criteria. After this screening, 49 papers remained.
3. **Introduction Review:** Some abstracts lacked adequate detail to determine relevance. In these instances, we reviewed the introduction sections to verify alignment with our research questions on test generation for microservices. Papers not explicitly addressing these topics were excluded, reducing the total to 47 papers.
4. **In-Depth Manual Review:** We conducted an extensive review of the remaining papers, emphasizing the methodology and results sections. Special attention was given to sources from SpringerLink due to limited filtering capabilities. Following this detailed assessment, 46 papers were confirmed as relevant.
5. **Snowballing [12]:** Finally, to identify additional relevant papers potentially missed in previous steps, we performed a backward snowballing search by reviewing the references of each selected paper. This process led to the inclusion of two more papers, bringing the final total of selected papers for our study to 48.

Table 7 summarizes how many papers from each database were ultimately included. This selection ensures that the final set of papers provides a high-quality and comprehensive dataset for addressing our research questions on microservices testing.

Table 7. Final set of selected papers.

Database	Number of Papers Selected
SpringerLink	10
IEEE Xplore	25
ACM Digital Library	8
Scopus	2
Web of Science	3
Total	48

2.3. Map

To systematically analyze existing research on test generation for microservices, we conducted a structured map study involving comprehensive data extraction, thematic categorization, and gap analysis. This process directly addresses our RQs by extracting critical data points such as methodologies, tools, metrics, and key findings from selected papers. The collected data were categorized into themes aligned with our research questions. These themes provide clear insights into prevalent techniques, practical applications, and impacts on system quality. The results support the identification of research gaps and future directions in the field.

2.3.1. Data Extraction

The data extraction stage systematically retrieved relevant information from selected papers, directly addressing our research questions. Here, "data points" refer to specific attributes of each paper that collectively build our evidence base for addressing the research questions. Table 8 contains the data points extracted from each selected paper.

Table 8. Detailed overview of data points extraction for test generation for microservices.

ID	Data Points	Description	Relevant Specific RQs
DP1	Publication Details	Authors, title, publication year, and venue.	-
DP2	Research Focus	The specific aspects of test generation for microservices addressed in the paper.	-
DP3	Methodology	The research methods and techniques used in the paper.	RQ1.1, RQ1.2, RQ1.4, RQ2.3
DP4	Tools and Frameworks	Specific tools or frameworks used for test generation mentioned in the paper.	RQ1.3, RQ1.4
DP5	Findings	Key results identified in the paper.	RQ1.4, RQ2.1, RQ2.2, RQ2.3, RQ2.4
DP6	Metrics	Performance metrics used to evaluate the quality of microservices tests.	RQ2.2, RQ2.4

We selected these six data points through iterative discussions among the authors, aiming to capture the most relevant information needed to answer the research questions and provide a comprehensive characterization of test generation in microservices. Here are details for each of them:

- **Publication details:** These give background information and the basic metadata about the paper, which helps to determine its relevance and credibility. These details also show how research is spread over time and among different venues, thereby indirectly pointing out major contributors or trends within a particular realm.
- **Research focus:** This provides clarity on the primary objectives and themes addressed in the paper, allowing us to ascertain its relevance and suitability for addressing our research questions.
- **Methodology:** This information explains how we conducted and implemented our investigations, helping us assess the robustness of each paper and its approach to testing. Understanding the methodology also supports the evaluation of techniques and challenges encountered during the testing phase (RQ1.1, RQ1.2, and RQ1.4). Additionally, it provides insight into the evaluation of test generation methods (RQ2.3).
- **Tools and frameworks:** These are essential for RQ1.3, which investigates the available tools and frameworks for test generation in microservices. This information helps in understanding the technological landscape and the practical implementations of test generation strategies.

- **Findings:** These are critical for all research questions, as they summarize the key results and contributions of each paper, helping to identify strengths and weaknesses (RQ1.4), the impact on quality(RQ2), and overall advancements in the field.
- **Metrics:** These are specifically relevant to RQ2.2 and RQ2.4. They focus on the evaluation criteria used to measure the effectiveness of test generation techniques. These criteria clarify how quality is quantified and how test coverage relates to test generation efficiency.

We used a standardized data extraction form to ensure consistency and accuracy. Any discrepancies were resolved through discussion.

2.3.2. Map Study

The map study involves categorizing and analyzing the extracted data to provide a comprehensive overview of current research on test generation for microservices. We derived each category from our research questions and the core focus areas that emerged during data extraction. The following steps detail our map study:

1. **Categorization:** We organized the extracted data into themes corresponding to RQ1 and RQ2. These themes represent recurring patterns in testing approaches, challenges, and evaluation methods. This process resulted in four primary categories: Testing Techniques, Tools, and Frameworks (C1); Systematic Study Contributions (C2); Impact on Quality (C3); and Metrics (C4). Table 9 presents these research categories, describes their scope, and aligns them with relevant research questions.
2. **Trend Analysis:** The study analyzed trends by examining the frequency of research in different areas and their publication sources. This analysis identified popular topics, emerging areas of interest, and underexplored aspects of microservices testing.
3. **Gap Identification:** We identified research gaps by analyzing areas with limited coverage. This step is essential for guiding future research on microservices test generation.
4. **Visualization:** To enhance comprehension of the findings, various visualizations, such as tables and graphs, were created. These visualizations summarize the distribution and focus of the selected papers concisely.

Table 9. Categorization of research areas in microservices test generation.

ID	Category	Description	Relevant RQs
C1	Testing Techniques, Tools, and Frameworks	We placed papers here if they described or evaluated specific methods, tools, or frameworks for microservices test generation. Examples include model-based testing, combinatorial testing, and automation tools.	RQ1.1, RQ1.3, RQ1.4
C2	Systematic Study Contributions	We included papers that provide systematic reviews, surveys, or mapping studies on microservices test generation. These synthesize multiple sources to present trends and insights.	RQ1
C3	Impact on Quality	We used this category for papers examining how test generation affects the quality of microservices-based systems. Topics here include strengths and weaknesses, empirical data on effectiveness, and how different testing methods detect defects.	RQ2
C4	Metrics	We assigned papers to this category if they proposed or assessed functional and non-functional metrics for microservices test generation. Such metrics include coverage, fault detection rate, and execution time.	RQ2.2, RQ2.4

3. Results

This systematic mapping study aimed to categorize and analyze various research contributions in the field of microservice testing. The distribution of papers across these categories is summarized in Figure 3. We classified each paper into a single primary category that best represented its main focus, even if it also discussed additional topics. Table 10 provides an overview of the distribution of papers within these categories and highlights the focus areas in microservice testing.

Table 10. Paper results of the mapping study.

ID	Title	Publisher	Year
Category: Testing Techniques, Tools, and Frameworks (C1)			
P1	A Reusable Automated Acceptance Testing Architecture for Microservices in Behavior-Driven Development [13]	IEEE	2015
P2	Automated Grey-Box Testing of Microservice Architectures [14]	IEEE	2022
P3	RAML-Based Mock Service Generator for Microservice Applications Testing [15]	SpringerLink	2017
P4	An architecture to automate performance tests on microservices [16]	ACM	2016
P5	An Automatic Test Data Generation Method for Microservice Application [17]	IEEE	2020
P6	Automatic Ex-Vivo Regression Testing of Microservices [18]	ACM	2020
P7	Automatic performance monitoring and regression testing during the transition from monolith to microservices [19]	IEEE	2019
P8	Automation of Regression test in Microservice Architecture [20]	IEEE	2018
P9	Benchmarks for End-to-End Microservices Testing [21]	IEEE	2023
P10	Challenges in Regression Test Selection for End-to-End Testing of Microservice-based Software Systems [22]	IEEE	2022
P11	A Microservice Regression Testing Selection Approach Based on Belief Propagation [23]	ACM	2023
P12	Design and Implementation of Intelligent Automated Testing of Microservice Application [24]	IEEE	2021
P13	Design and Research of Microservice Application Automation Testing Framework [25]	IEEE	2019
P14	Design, implementation, and testing of a microservices-based Digital Twins framework for network management and control [26]	IEEE	2022
P15	Efficient software test management system based on microservice architecture [27]	IEEE	2022
P16	Fault Diagnosis for Test Alarms in Microservices through Multi-source Data [28]	ACM	2024
P17	JoT: A Jolie Framework for Testing Microservices [29]	ACM	2023
P18	Learning-Based Testing of Distributed Microservice Architectures: Correctness and Fault Injection [30]	ACM	2015
P19	Microservices Integrated Performance and Reliability Testing [31]	IEEE	2022
P20	Microusity: A testing tool for Backends for Frontends (BFF) Microservice Systems [32]	IEEE	2023
P21	Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models [33]	IEEE	2022
P22	UTEMS: A Unit Testing Scheme for Event-driven Microservices [34]	IEEE	2023
P23	Solving the Instance Identification Problem in Microservice Testing [35]	SpringerLink	2021
P24	Crunch: Automated Assessment of Microservice Architecture Assignments with Formative Feedback [36]	SpringerLink	2018
P25	White-Box Fuzzing RPC-Based APIs with EvoMaster: An Industrial Case Study [37]	Scopus	2023
P26	RESTful API automated test case generation [38]	Scopus	2017
P27	NxtUnit: Automated Unit Test Generation for Go [39]	Web of Science	2023
P28	A Discrete Dynamic Artificial Bee Colony with Hyper-Scout for RESTful web service API test suite generation [40]	Web of Science	2021
P29	Gremlin: Systematic Resilience Testing of Microservices [41]	IEEE	2016

Table 10. *Cont.*

ID	Title	Publisher	Year
Category: Systematic Study Contributions (C2)			
P30	An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study [42]	IEEE	2024
P31	Contract Testing in Microservices-Based Systems: A Survey [43]	IEEE	2023
P32	Testing Microservices Architecture-Based Applications: A Systematic Mapping Study [44]	IEEE	2020
P33	Overview of Information System Testing Technology Under the "CLOUD + Microservices" Mode [45]	SpringerLink	2022
Category: Impact on Quality (C3)			
P34	A Quantitative Approach for the Assessment of Microservice Architecture Deployment Alternatives by Automated Performance Testing [46]	SpringerLink	2018
P35	Analysis for Microservice Architecture Application Quality Model and Testing Method [47]	IEEE	2023
P36	Assessing Black-box Test Case Generation Techniques for Microservices [48]	SpringerLink	2022
P37	Comparison of Runtime Testing Tools for Microservices [49]	IEEE	2019
P38	Consumer-Driven Contract Tests for Microservices: A Case Study [50]	SpringerLink	2019
P39	Fitness-guided Resilience Testing of Microservice-based Applications [51]	IEEE	2020
P40	Inferring Relations Among Test Programs in Microservices Applications [52]	IEEE	2021
P41	Microservice-tailored Generation of Session-based Workload Models for Representative Load Testing [53]	IEEE	2019
P42	Microservices: A Performance Tester's Dream or Nightmare? [5]	ACM	2020
P43	Research on Microservice Application Testing Based on Mock Technology [54]	IEEE	2020
P44	Research on Microservice Application Testing System [55]	IEEE	2020
P45	Research on Performance Optimization Method of Test Management System Based on Microservices [56]	IEEE	2023
P46	Search-Based Performance Testing and Analysis for Microservice-Based Digital Power Applications [57]	IEEE	2023
P47	Zero-Config Fuzzing for Microservices [58]	Web of Science	2023
Category: Metrics (C4)			
P48	End-to-End Test Coverage Metrics in Microservice Systems: An Automated Approach [59]	SpringerLink	2023

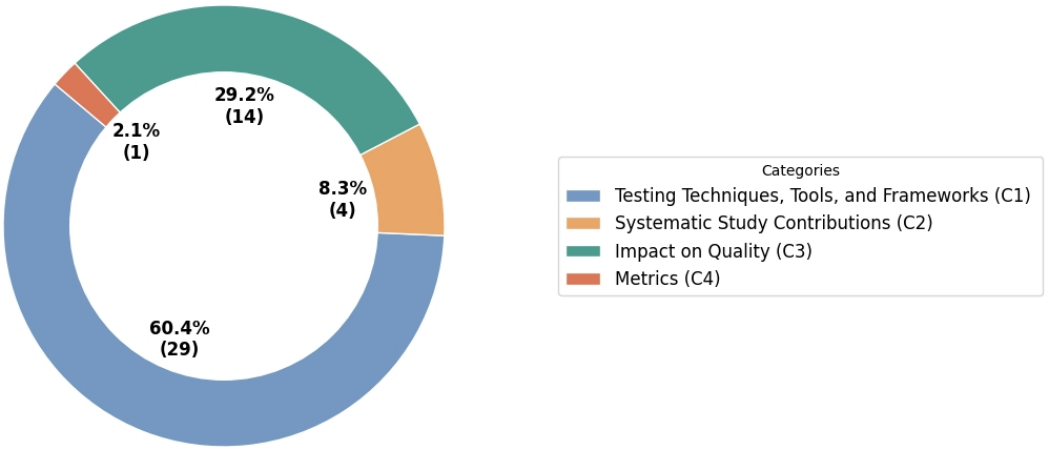


Figure 3. Paper distribution based on categories.

3.1. Trends in Microservice Testing Publications

This subsection presents an analysis of publication trends in microservice testing papers from 2015 to 2024. Figure 4 illustrates the trend of the total number of publications during this period and highlights several notable shifts.

Early research output from 2015 to 2017 was minimal and reflected the nascent stage of microservice architectures. The number of publications increased significantly from 2019

onward, which aligns with greater industry adoption of microservices and the resulting demand for comprehensive testing methodologies. The 2023 peak, representing 25% of all sampled papers, indicates a strong research focus and a dedicated effort to enhance verification practices as microservices mature.

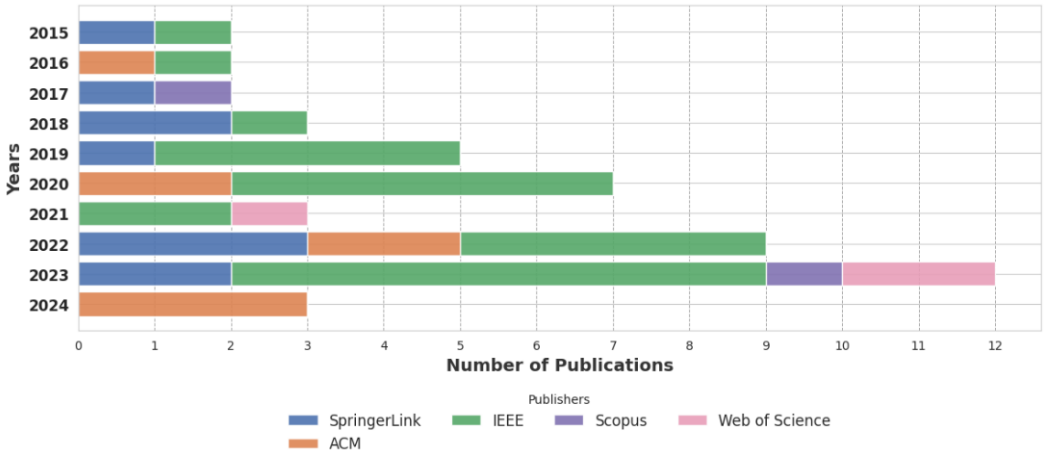


Figure 4. Publications per year by publisher.

A substantial share of work in 2021 (6.25%) occurred with the onset of the COVID-19 pandemic, a time when many organizations accelerated their digital transformation and adopted cloud-native approaches. In 2024, the data shows a low number of publications. This is expected since the mapping study was completed in the first half of the year, and data collection did not cover the entire year. This limitation contributed to the lower publication count. Overall, these patterns highlight growing scholarly engagement with microservice testing. Future research could build upon these findings by integrating systematic inquiry and establishing benchmarks for assessing testing efficacy in distributed systems.

3.2. General Quantitative Trends of Microservice Testing

This section presents the overall quantitative patterns observed among the selected papers. Figure 4 (Section 3.1) already illustrates the annual growth in publications, but further insights emerge when examining how research efforts are distributed across categories:

- **Testing Techniques, Tools, and Frameworks (C1):** A total of twenty-nine papers (60.4% of the final set) concentrate on designing or enhancing specialized testing methods, and automation frameworks tailored to microservices. This prevalence underscores the community's focus on practical, hands-on solutions that address testing challenges in distributed services.
- **Systematic Study Contributions (C2):** Only four papers (8.3%)—P30 [42], P31 [43], P32 [44], and P33 [45]— provide surveys, reviews, or mapping studies. Although limited in number, these papers offer valuable syntheses of existing knowledge and identify significant gaps.
- **Impact on Quality (C3):** Fourteen papers (29.2%) explore how various test generation strategies influence system reliability, fault tolerance, and performance.Their empirical findings guide practitioners in strengthening microservice architectures.
- **Metrics (C4):** Only one paper (2.1%), P48 [59], explicitly targets metrics for assessing test effectiveness or performance in microservices. This scarcity of publications indicates a need for standardized measurement approaches. Such standardization would benefit practitioners seeking rigorous comparisons of testing methods.

Overall, the distribution indicates a strong interest by researchers in creating new testing solutions (C1) and evaluating how these solutions affect microservice robustness

(C3). The comparatively low number of published systematic studies (C2) and metrics-related works (C4) highlight opportunities for future contributions, especially in developing comprehensive reviews and well-defined metrics frameworks. This analysis confirms the community's emphasis on solving real-world testing problems and reveals underexplored areas.

4. Approaches and Challenges (RQ1)

RQ1 focuses on the key approaches and challenges in test generation for microservices. The distributed nature, independent deployments, and frequent updates of microservices create unique testing complexities. Effective strategies must address challenges such as service interactions, scalability, fault tolerance, and automation. This section explores the primary testing methods, their limitations, and the critical factors influencing microservices test generation.

4.1. Prevalent Testing Techniques (RQ1.1)

In this subsection, we analyze the predominant testing techniques employed in microservice architectures. These approaches play a crucial role in maintaining functionality and resilience within rapidly evolving distributed systems.

Table 11 summarizes each technique, explains its focus, and lists representative references. These particular references were chosen because they exemplify how each technique is applied in practice. For example, some works focus on developing novel frameworks or tools (e.g., [48]), while others assess real-world efficiency or scalability under testing (e.g., [16,46]). These papers illustrate the breadth of testing approaches and their key contributions to microservices testing research.

Table 11. Summary of primary testing techniques and key references.

ID	Technique	Focus or Objective	Key References
TE1	Unit Testing	Validates individual microservices at a fine-grained level. Ensures correct functionality in isolation.	[34,39]
TE2	Regression Testing	Checks that recent updates do not break existing features.	[18–20,22,23]
TE3	Grey-Box Testing	Combines black- and white-box insights. Uses partial knowledge of internal structures and service interactions.	[14]
TE4	Acceptance Testing	Validates overall behavior against user requirements. Uses natural-language or BDD approaches.	[13]
TE5	Performance/Load Testing	Evaluates scalability and robustness under stress conditions. Identifies bottlenecks in distributed services.	[16,21,46]
TE6	Contract Testing	Verifies service interfaces and consumer-driven contracts. Ensures compatibility across independent services.	[33,43,50]
TE7	Fault Injection/Resilience	Simulates service failures. Tests error-handling, fallback mechanisms, and overall fault tolerance.	[30,41,51]
TE8	Fault Diagnosis	Identifies root causes of failures in interconnected microservices. Facilitates targeted debugging.	[28]
TE9	Test Case/Data Generation	Automatically produces test inputs or scenarios. Aims to increase coverage and reveal edge cases.	[15,17,37,48]

Figure 5 demonstrates that performance and load testing (TE5) are the most prevalent methods. This trend reflects the necessity of verifying service scalability under intensive operational conditions. Additionally, the significant emphasis on test cases and data generation (TE9) highlights efforts within the research community to systematically handle

diverse input scenarios. These observations indicate that rigorous validation strategies are essential for ensuring reliability in distributed microservice environments. Effective testing techniques enable individual services to manage high loads, achieve seamless integration, and detect latent defects before deployment. Without comprehensive testing, unexpected bottlenecks or functional deficiencies could compromise the quality and stability of microservice-based applications.

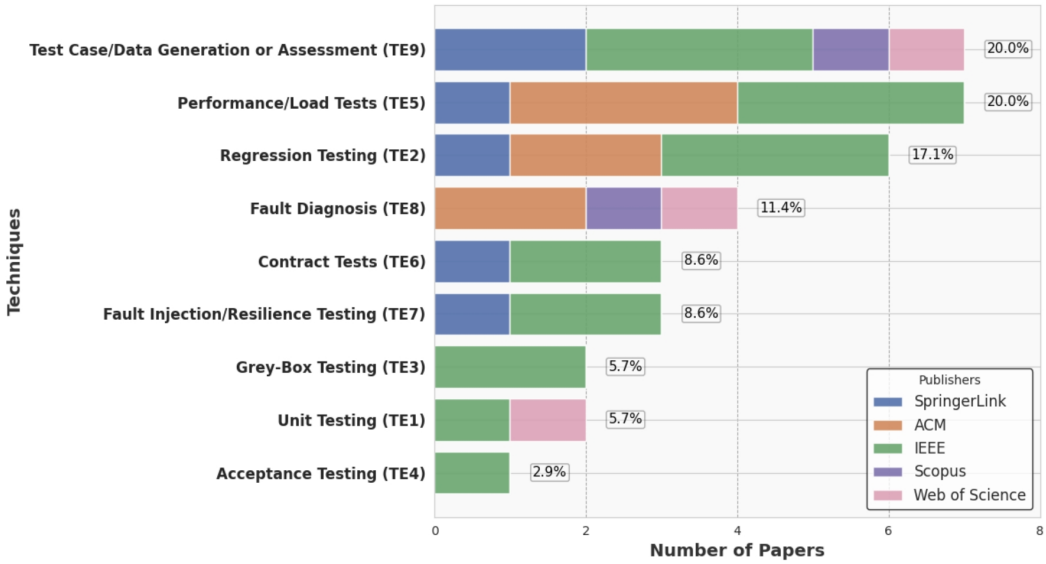


Figure 5. Distribution of papers across testing techniques for microservices.

Moreover, these techniques are prevalent in microservices testing for several reasons that distinguish them from older Service-Oriented Architecture (SOA) [60] or monolithic contexts. For instance, the independent deployability of microservices amplifies the need for automated regression (RQ1.1 and RQ1.4), as each service changes at its own pace. Similarly, techniques like contract testing (TE4) (e.g., [33,43,50]) are more critical in microservices than in monolithic or traditional SOA systems because frequent versioning and service decoupling demand explicit interface agreements. In contrast to classical SOA, microservices emphasize lightweight protocols and ephemeral instances, further prompting the use of container-based performance/load testing to account for dynamic scaling and short-lived deployments [16,21,46]. These factors collectively explain why performance, contract, and regression testing appear more prominently in microservices papers.

Building on this analysis, the grouping of papers highlights the interdisciplinary nature of the previously mentioned techniques in microservices testing. One paper can belong to multiple testing technique categories, demonstrating the interdisciplinary nature of microservice testing. Our thorough analysis reveals a balanced focus on functional and non-functional aspects, thus ensuring holistic validation of microservices.

Beyond grouping, a deeper examination reveals patterns in the co-occurrence of techniques. We observed that techniques such as performance/load testing (TE5) and regression testing (TE2) frequently co-occur. This indicates a trend that ensuring performance often goes hand in hand with verifying that no regressions appear during continuous deployment. Similarly, acceptance testing (TE4) is closely associated with contract testing, demonstrating that behavior verification and API-level contracts often share overlapping goals in guaranteeing consistency across service boundaries.

Figure 6 contains a comparative look at the papers written on functional vs. non-functional testing. Of the 20 papers that have been published regarding functional testing, this indicates the necessity of thoroughly operationalizing microservices. Some examples of

functional testing approaches are unit, black-box, and acceptance tests that determine whether each microservice satisfies its requirement specification and works with other services.

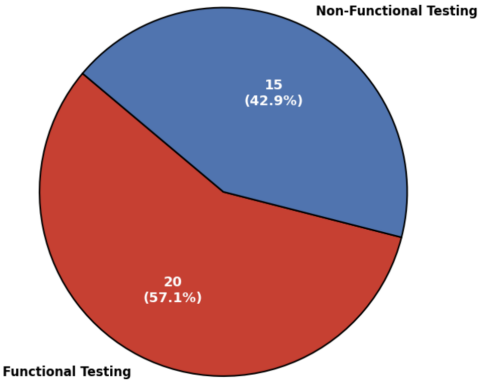


Figure 6. Functional vs. non-functional testing papers.

On the other hand, 15 papers focus on non-functional testing, addressing performance, security, resilience, and other critical aspects. These papers highlight the importance of evaluating a system's robustness within its operational environment, particularly in the context of microservices. The system must undergo performance and load tests, fault injection, and resilience tests to ensure it can handle loads and remain secure after intentional faults. These evaluations provide confidence that all the services involved meet the necessary requirements. The combination of functional and non-functional testing covers a critical spectrum in microservices validation. This is because the correctness of operations and the overall quality attributes of the system have been established and ensured.

4.2. Commonly Adopted Testing Approaches (RQ1.1)

The preceding analysis (Table 11) details primary testing techniques identified across microservice architectures. However, within these techniques, several testing approaches are commonly adopted to address specific challenges unique to microservices. Each technique often spans multiple approaches, providing methodological foundations applicable across various testing contexts. Additionally, certain widely adopted testing approaches identified in the literature, though not explicitly grouped in Table 11, frequently complement the listed techniques. These additional approaches, such as black-box testing, cloud-based testing techniques, and instance identification, further demonstrate the breadth of methodological solutions researchers have utilized to address diverse microservice-specific testing challenges. This expanded view acknowledges the interdisciplinary and complementary nature of approaches and their methodological overlap with the primary techniques outlined previously.

- **Unit Testing for Event-Driven Microservices (TE1):** Event-driven architectures introduce asynchronous complexities, necessitating specialized unit testing approaches. Paper [34] proposes UTEMS, explicitly designed to validate the correctness of event-driven microservices under various event-trigger scenarios, thereby addressing complexities unique to asynchronous service communication.
- **Regression Testing (TE2):** Due to frequent updates that are inherent in microservice systems, automated regression testing is essential to quickly identify regressions caused by continuous deployment. Papers [18–20] present automated frameworks designed explicitly for regression testing. Paper [18] focuses on ex-vivo regression testing in isolated microservice environments, while papers [19,20] emphasize regression testing automation for detecting feature regressions efficiently, especially under rapid deployments.

- **Black-box and grey-box testing (TE3):** Black-box testing assesses external interfaces without internal knowledge, whereas grey-box testing incorporates partial insights into the internal structure. Paper [48] demonstrates a black-box approach (uTest) that systematically covers functional scenarios, and Paper [14] details MACROHIVE, a grey-box approach integrating internal service-mesh insights to achieve deeper coverage during testing.
- **Acceptance Testing (TE4):** Behavior-Driven Development (BDD) facilitates clear, user-centric specification and acceptance criteria. It bridges communication between developers and non-developers through natural-language scenarios. Paper [13] exemplifies this by introducing a reusable automated acceptance testing architecture leveraging BDD for enhanced clarity and maintainability in microservices.
- **Performance and Load Testing (TE5):** Performance and load testing techniques address the crucial aspect of scalability and stability under high operational stress in microservices. Papers [46], [16], and [21] detail various methods, from quantitative assessments of deployment alternatives ([46]) to automated architectures specifically tailored to microservice performance testing ([16]). Paper [21] further emphasized end-to-end benchmarking to capture realistic, whole-system performance characteristics rather than isolated service behavior alone.
- **Contract Testing (TE6):** Contract testing is crucial due to independent deployments and frequent changes in microservice interfaces. Papers [50], [43], and [33] illustrate how contract testing approaches can ensure reliable interactions by explicitly validating API agreements. Paper [33] specifically integrates consumer-driven contracts with state models, enhancing the accuracy and consistency verification across microservices.
- **Fault Injection and Resilience Testing (TE7):** Fault injection and resilience evaluation techniques verify the system's ability to operate effectively despite faults and disruptions. Papers [51], [30], and [41] exemplify controlled fault-simulation scenarios aimed at testing fault tolerance. For instance, paper [51] introduces fitness-guided resilience testing, systematically identifying weaknesses to guide resilience improvements.
- **Fault Diagnosis (TE8):** Fault diagnosis identifies root causes of failures within interconnected microservices, facilitating targeted debugging. Paper [28] specifically addressed fault diagnosis using multi-source data to pinpoint the precise cause of microservice disruptions, facilitating targeted debugging and quicker resolution.
- **Test Case/Data Generation (TE9):** Automated test case and data generation aims to improve coverage and efficiency while revealing complex edge cases in microservices testing. Papers [17], [48], and [15] demonstrate different approaches, from automatic data generation techniques for microservice scenarios to RAML-based mock service generators, emphasizing automation's role in handling diverse input scenarios.
- **Static and Dynamic Analysis:** These analyses uncover code- and runtime-level issues. Papers [17,24,42] demonstrate static code checks to catch early defects and dynamic monitoring to observe real-time performance. Paper [24] employs both methods to enhance test effectiveness in microservice applications.
- **API Testing and Tracing:** Detailed request tracing can reveal hidden integration flaws. Paper [32] proposes Microusity, a specialized tool for RESTful API testing and visualization of error-prone areas. Paper [37] extends EvoMaster with white-box fuzzing for RPC-based APIs, uncovering security weaknesses.
- **Cloud-Based Testing Techniques:** As microservices often run in cloud environments, specialized solutions have emerged. Paper [45] presents methods to leverage automated provisioning and large-scale simulations, ensuring microservice deployments

can scale and remain reliable. Paper [26] applies such ideas to Digital Twins, underscoring the practical breadth of cloud-based testing.

- **Instance Identification and Resilience Evaluation:** Identifying distinct microservice instances helps testers monitor localized failures. Paper [35] offers a method to isolate specific components, clarifying root causes and streamlining resiliency checks.

The approaches mentioned above were specifically highlighted due to their recurring presence and effectiveness in addressing prevalent microservice-specific challenges, such as rapid deployments, distributed interactions, and frequent version changes. They integrate naturally with techniques summarized earlier in Table 11, reinforcing the depth and interdisciplinary nature of prevalent microservice testing research.

4.3. Techniques Comparison with Monolithic Applications (RQ1.2)

Testing strategies for microservices are distinctly different from those used in traditional monolithic applications because of the unique architectural features and operational needs of microservices. Table 12 presents the key contrasts that influence testing approaches. Each row specifies a testing dimension, referring to a critical aspect of the testing process, such as granularity, deployment, or fault tolerance, and its corresponding architectural differences.

Table 12. Key differences between monolithic and microservices testing.

Dimension	Monolithic Focus	Microservices Focus
Granularity and Isolation	All functionalities reside in a single deployment. Tests often target the entire system at once [61].	Independent services run in their own contexts. Each service undergoes isolated unit and integration testing. This prevents cascading failures [13,18].
Deployment Environment	Stable, homogenous configurations. Fewer variations in test scenarios.	Dynamic, containerized settings [16,46]. Testing tools handle multiple orchestrated services with varied resource allocations.
Continuous Integration	A single artifact updates the whole application. Regression tests can be slower and less frequent [1,61].	Rapid releases demand automated checks after each update. Regression testing quickly confirms no new defects [19,20].
Inter-Service Communication	Internal calls happen within one codebase. Communication stays straightforward.	Services rely on lightweight APIs and evolving contracts. Consumer-driven contract tests reduce integration issues [33,43,50].
Fault Tolerance and Resilience	Errors may propagate [62] but remain in a single codebase. Traditional chaos testing sees limited use [63].	Distributed faults require resilience tests and fault injection [30,41,51]. Network delays and partial failures often occur.
Performance and Scalability	Focus on overall throughput or response time [1]. One large codebase handles vertical scaling [62].	Each service scales independently, but network overhead affects performance. Tests target latency and load across components [21,46].
Testing Tools and Frameworks	Classic frameworks (e.g., JUnit [64]) check end-to-end behaviors in one environment.	Specialized tools handle contract tests, mock services, and asynchronous flows [15,29]. Environments evolve faster.

Furthermore, monolithic systems benefit from straightforward communication paths and fewer environment configurations [61]. Testing teams often run large-scale integration tests for the entire application. In contrast, microservices require specialized tools to manage versioned interfaces and dynamic scaling [33,43,50]. Failures can cascade across multiple services; thus, testers utilize consumer-driven contract testing, automated regression suites, and fault injection to maintain reliability [30,51]. By focusing on each service boundary and its interactions, teams can mitigate the risks introduced by frequent deployments and distributed dependencies.

4.4. Test Generation Techniques (RQ1.3)

The exploration of test generation in microservices with respect to tools and frameworks often overlaps with broader testing techniques, as discussed in RQ1.1. RQ1.3 focuses more specifically on dedicated tools and frameworks for generating tests or test cases

and their roles within the microservices landscape. Table 13 summarizes key solutions, highlighting each tool or approach and its main focus.

Table 13. Key tools and frameworks for test generation in microservices (RQ1.3).

ID	Tool/Framework	Focus or Method	References
TF1	JoT	Jolie-based framework targeting service-oriented testing for microservices.	[29]
TF2	RAML-based Mock Gen.	Generates mock services from RAML specs for integration and end-to-end testing.	[15]
TF3	EvoMaster (Extended)	Uses white-box fuzzing and evolutionary algorithms to improve coverage and detect flaws.	[37]
TF4	RESTful API Test Tool	Applies evolutionary computation for integration test generation and optimizes fault detection.	[38]
TF5	NxtUnit	Automates test generation for Go microservices using static and dynamic analysis.	[39]
TF6	AI-Driven Algorithm	Selects and optimizes test cases based on coverage and efficiency metrics.	[40]
TF7	Ex-Vivo Regression Testing	Detects regressions using isolated systems, replicating real environments.	[18]
TF8	Search-Based Perf. Testing	Generates performance test cases under varying parameters to identify bottlenecks.	[57]
TF9	uTest	Uses a pairwise combinatorial approach for black-box test generation via OpenAPI.	[48]

While testing tools for microservices have significantly advanced, dedicated techniques for generating test cases require additional development. Most of the available tools are directed towards generating the test environment or generating tests, especially for regression testing. This stands in stark contrast to the testing landscape of monolithic systems, where there is a plethora of tools and methodologies aimed at creating automated test cases. The scarcity of dedicated test case generation tools for microservices indicates an opportunity for further research to improve testing efficiency.

4.5. Strengths and Weaknesses (RQ1.4)

BDD and Acceptance Testing effectively bridge the communication gap between technical and non-technical stakeholders. Defining test criteria using natural language ensures that all parties share a common understanding of system behavior under specific conditions. This inclusive approach is exemplified in [13], where BDD frameworks facilitate the creation of comprehensive and understandable acceptance tests. However, BDD is only effective when behaviors and acceptance criteria are defined precisely. Misunderstandings or incomplete definitions can create test coverage gaps, increasing the risk of undetected defects. AI-driven test case generation, such as in [40], enhances efficiency by reducing redundant test cases while maintaining high coverage levels. However, it introduces increased computational overhead, making it less practical for large-scale test executions.

Black-box testing [48] is strong because it does not require internal knowledge of the system when testing microservices. This makes it easy and flexible for functional and robustness tests. A representative case of this is the uTest tool, which generates efficient test cases with good fault-detection capabilities. However, black-box tests may miss internal interactions between microservices and dependencies, leading to failures. The grey-box testing, like the MACROHIVE strategy mentioned in [14], improves the detection of internal faults as well as produces detailed coverage information that may be missed by just using black-box testing. The major strength of the grey-box approach is its wide-ranging internal and external cover, thus making it effective in revealing complex failure scenarios. It could, however, prove more difficult to implement since both perspectives are needed (internal and external) and may involve sophisticated infrastructure, such as service mesh, for monitoring interactions among different services. On the other hand, white-box testing, as implemented in EvoMaster [37], significantly improves security testing and API robustness

but requires deep access to internal service structures, which may not always be feasible in cloud-based deployments.

The primary advantage of regression testing is its role in preventing new changes from introducing regressions into existing functionality. Automated regression testing frameworks, for example, those explained in [20], allow code changes to be quickly verified, thereby reducing the risk of new defects. The method in [18] uses isolated environments to detect regressions and enhance system reliability. However, this approach can be resource-intensive and challenging to maintain for large codebases. It can be tedious to maintain and update a comprehensive set of regression tests, which could require significant computational resources, as discussed in [19].

Methods for performance and load testing are crucial for assessing the scalability and robustness of microservices under various load conditions. The quantitative approach for assessing microservice architecture deployment alternatives, as seen in [46], and the end-to-end performance benchmarking discussed in [21] are key examples. These methods identify bottlenecks and performance degradation points, ensuring that the system handles peak loads efficiently. However, performance and load testing can be challenging to implement accurately due to the need for realistic and representative test conditions. Replicating realistic test conditions requires significant effort and remains challenging [16].

Contract testing provides a reliable means of making sure that services conform to predefined interaction contracts, hence preventing integration issues from arising. The consumer-driven contract testing approach, detailed in [50], validates that services interact correctly by validating their agreements. This method's strength lies in the fact that it detects integration failures early enough during the development stage, thereby reducing the chances of system-wide failure occurrence. The effectiveness of contract testing depends on how well contracts are defined. Any oversight or lack of detail can lead to undetected issues, as highlighted in [43].

Fault injection and resilience testing are crucial for evaluating microservices' fault tolerance. The techniques discussed in [51] and [30] intentionally introduce faults to assess system responses, identifying failure points and enhancing robustness. Similarly, Gremlin [41] systematically injects network-level faults into microservice communications, enabling rapid testing and precise validation of resilience patterns without altering the microservices' code. However, creating realistic fault scenarios remains challenging, potentially causing inaccurate assessments of a system's resilience [30].

Answer to RQ1: Key Approaches and Challenges in Test Generation for Microservices

Our analysis reveals that microservice testing (RQ1.1) commonly utilizes performance/load testing, contract testing, regression testing, and fault injection. These methods address challenges arising from distributed interactions and frequent updates. Compared to monolithic systems (RQ1.2), microservices introduce distinct testing needs, including interface evolution, high concurrency, and advanced fault isolation. Frameworks such as JoT and EvoMaster (RQ1.3) facilitate automated testing but face limitations with dynamic, container-based deployments. Each technique (RQ1.4) involves trade-offs: contract testing enables early regression detection but requires precise contract definitions, whereas grey-box and white-box testing achieve deeper coverage at the expense of increased complexity. These findings emphasize the importance of adaptive testing strategies and efficient resource management to sustain reliability and scalability in microservice architectures.

5. Impact on the Quality of Testing (RQ2)

In a dynamic and distributed environment such as microservices-based systems, testing must be conducted effectively to achieve high quality. This section focuses on testing or test generation and how it affects these two aspects of microservices to answer RQ2. Therefore, we aimed to establish how testing practices can be optimized for microservices by considering different techniques used in generating tests and their impact on system quality and dependability.

In this section, we observed that quality-driven testing in microservices often combines coverage metrics with advanced techniques like fault injection to validate resilience. There is also a growing trend toward integrating performance testing with regression suites; hence, RQ2 explores how these intertwined factors contribute to robust microservices testing. To clarify how our findings address RQ2, we summarized the key insights in Table 14. This table highlights how each component influences the overall testing process and shows all the key findings.

Table 14. Summary of RQ2 findings: Factors, metrics, and impact on quality.

Aspect	Key Observations
RQ2.1: Factors	Interaction chain inconsistency [5] Service independence vs. inter-dependencies [33,43,50] Dynamic environments (cloud-native) [45] Complex regression test selection (E2E) [22]
RQ2.2: Metrics	Endpoint coverage metrics [59] Functional (e.g., parameter coverage) [14,48] Non-functional (response time, throughput) [16,46]
RQ2.3: Enhancing Quality	Mock services for consistent integration tests [15] Early defect detection with grey-box testing [14] Performance and resilience testing synergy [30,41,51] CI/CD-driven automation [20,25]
RQ2.4: Coverage vs. Efficiency	Comprehensive coverage often conflicts with test generation speed Mock services can improve coverage while controlling complexity [15] Balancing coverage and resource usage is critical [5,18,20]

5.1. Factors Influencing the Testing (RQ2.1)

The complexity and effectiveness of testing in microservice architectures are affected by several factors that contribute to this complicated process. This subsection focuses on RQ2.1 by introducing the primary factors that affect microservices testing.

The inconsistency of the interaction chain or inter-service communication chain is one major problem that was identified in [5]. This refers to how services interact with each other across different deployments and instances, which may lead to unexpected behavior and difficulty in ensuring reliable communication between services. Their illustrative scenario shows how services might deviate from the standard interaction path due to several factors:

- Dynamic Service Discovery: Microservice architectures rely heavily upon dynamic service discovery mechanisms, where services find each other during runtime. Depending on the current state of the system and network conditions, various interaction patterns may arise from this process.
- Load Balancing: Incoming requests are distributed over multiple service instances by load balancers to utilize resources optimally while ensuring availability. However, due to traffic routing through different instances by the load balancer, there might be inconsistency within the service interaction chain.
- Service Versioning: Frequent updates are made to microservices, whereby new versions are deployed. Thus, when more than one version is running concurrently,

interactions may differ based on feature set compatibility across versions, which leads to inconsistencies.

- Failure and Recovery: Failure and recovery events can occur unpredictably within service instances. During such events, communication paths break down temporarily, and the sequence of interactions changes as the system reconfigures itself to handle the failure.

These factors make it difficult to maintain a consistent and predictable interaction chain in microservice environments, thereby complicating the accurate replication of real-world scenarios for testing purposes. Ensuring microservice reliability remains a significant challenge for both developers and testers.

Service independence is another crucial aspect, as each microservice functions independently, so the testing strategies must ensure that changes occurring in one service do not negatively affect others. Refs. [33,43,50] propose consumer-driven contract testing to manage and verify service interactions. This approach strengthens testing without compromising service independence.

Dynamic environments and frequent deployments add further complexity to microservices testing. Since these systems are developed using agile methodology, where services are continuously updated and redeployed, there is a need for effective regression testing methods that can quickly identify and address issues introduced by new changes. Refs. [18–20] emphasize the importance of automated regression testing in maintaining system reliability during frequent updates.

The **deployment context** of microservices, often in cloud-native environments, introduces unique testing challenges. Ref. [45] discusses how cloud-based environments influence microservice quality. This is achieved by leveraging the scalability and flexibility offered by the cloud with regard to simulating real-world scenarios during different loads and stress conditions. The handling ability of various microservices can be tested effectively under such circumstances.

The **deployment of microservices** for testing purposes is one factor related to deployment. In setting up the test environment, Docker is commonly used among other tools, and it often receives updates and may modify configurations, which means that testers must locate a compatible version of Docker or update all the configurations to align with the latest updates for Docker for each microservice. Previous experiments demonstrated significant challenges with this approach, hence the need for careful configuration management to have a reliable and consistent test environment.

The challenge of **regression test selection** in E2E testing scenarios, as highlighted in [22], is also one factor. Because they are independent and quite often have loose coupling between them, microservices make it difficult to determine which subset of tests should be re-executed after any changes occur. The reason behind this complexity is that modifications made in one service may affect many others, leading to uncertainty about which tests are needed to ensure the integrity of the system. Therefore, efficient methods of selecting regression tests must avoid redundant work while at the same time preventing new faults caused by updates from being detected. Hence, Ref. [22] stresses the need for advanced selection strategies capable of accurately identifying change impact scope within microservice architecture, thus optimizing process regression testing and preserving system reliability.

In summary, the testing of microservices is influenced by the inconsistency of the interaction chain, the independence of services, dynamic and frequent deployments, and the specific challenges of cloud-native environments. These factors necessitate sophisticated and adaptive testing strategies to ensure the quality of microservices-based systems.

5.2. Metrics for Evaluating the Quality of Tests (RQ2.2)

Metrics play a crucial role in evaluating the quality and effectiveness of microservices tests. These metrics primarily measure functional coverage, identify untested areas, and assess the reliability and robustness of microservices.

Ref. [59] introduces three key metrics for evaluating endpoint coverage in end-to-end (E2E) test suites for microservices. These include Test Case Endpoint Coverage, Complete Test Suite Endpoint Coverage, and Microservice Endpoint Coverage. These metrics assess the extent of E2E test suite coverage in microservices, providing insights into the completeness of the testing process.

Microservice Endpoint Coverage defines the percentage of all possible endpoints within every single microservice that has already been tested. This metric determines whether a service has been adequately tested. For instance, if there are 10 endpoints in one given microservice but only 8 have been covered by tests, then Microservice Endpoint Coverage would stand at 80%.

Test Case Endpoint Coverage measures the proportion of endpoints covered by a single test case relative to the total system endpoints. It indicates the efficiency or effectiveness levels of particular tests while they attempt to cover different parts of any given application being tested (here, represented as its endpoints, otherwise known as APIs).

Complete Test Suite Endpoint Coverage measures the overall coverage rate achieved through executing all available tests against an API set provided by some service provider during the validation phase, which forms part of its life cycle management activities according to rules laid out within it around those stages themselves, like design time visibility into services' interfaces. Such coverage can also be analyzed from other perspectives, such as the holistic viewpoint, where we saw them covering whole functionalities offered by such systems.

Along with the use of the metrics in Ref. [59], we identified both functional and non-functional metrics that are critical for assessing the overall quality of microservices testing in our research. These metrics, shown in Table 15, have been adapted from various sources to ensure comprehensive coverage and the resilience of the tests. Functional metrics like Path Coverage, Operation Coverage, and Status Code Coverage enable us to know how the tests dig into microservice functions best; this is according to Refs. [14,48], which state that these kinds of measures help achieve thorough validation for service operations by making sure every possible path/state is covered during testing.

Moreover, it should be noted that while there are more non-functional than functional measurements identified by our study, they may not capture everything about them, as is expected due to their nature, which is influenced less by the architectural complexity involved with these small-sized programs. The reason why this predominance exists lies in the fact that most non-functionals, such as Response Time, Throughput, or Fault Tolerance, can work, no matter what kind of a system it is – monolithic or based on microservices.

These metrics in Table 15 demonstrate how intricate performance evaluations can become within environments characterized by multiple services, each providing particular functionality individually over network connections. This does not mean that such indicators do not change depending upon design parameters, etc., but rather they are applied universally, regardless of where anything else happens within or without any given organization's information technology setup. However, while we acknowledge the need for many-sidedness when dealing with matters related to service-oriented computing systems, reliability aspects were mostly ignored, thus leaving out functional perspectives in our research so far.

Table 15. Functional and non-functional metrics.

Functional Metrics	
Coverage Metrics	
Path Coverage [14,48]	Operation Coverage [48]
Parameter Coverage [48]	Parameter Value Coverage [48]
Request Content-Type Coverage [48]	Status Code Class Coverage [14,48]
Status Code Coverage [14,48]	Response Content-Type Coverage [48]
Microservice Endpoint Coverage [59]	Test Case Endpoint Coverage [59]
Complete Test Suite Endpoint Coverage [59]	
Test Execution Metrics	
Avg. Number of Executed Tests [48]	Avg. Failure Rate [14,48]
Error Coverage Rate [17]	Count of Generated Test Data [17]
Structural Metrics	
Coupling (MPC) [47]	Cohesion (LCOS) [47]
Traceability [47]	
Non-Functional Metrics	
Performance Metrics	
Response Time [5,16,18,20,21,30,36,46,53,56,57]	Throughput [16,53]
Scalability [46,47]	Availability [47]
Fault Tolerance [36,47]	Resource Usage Rate [5,20]
System Failure Rate [20]	Performance Thresholds [19,20,53]
Usage Intensities [18]	CPU Utilization [53]
Memory Consumption [53]	
Failure Metrics	
Failed Requests (FR) [31]	Connection Errors Ratio (CE) [31]
Server Errors Ratio (SE) [31]	Internal Microservice Coverage [14]
Dependency Coverage [14]	Propagated & Masked Failures [14]
Timing Metrics	
Start-Up Time [22]	Runtime Overhead [22]
Execution Time [49,57]	Trace Coverage [57]
Performance Degradation (PD) [31]	

Our study identified a greater number of non-functional metrics than functional metrics, reflecting their general applicability across system architectures, whether monolithic- or microservice-based. Non-functional metrics such as Response Time, Throughput, and Fault Tolerance remain relevant, regardless of underlying architectural complexity. Nonetheless, these metrics may not fully capture the nuanced impacts of architectural choices inherent in microservices. Furthermore, despite the detailed metrics proposed, our analysis identified a notable gap regarding dedicated test case generation tools specifically designed for microservices. Existing frameworks, like JoT and EvoMaster, primarily support automation in test environment setups but seldom fully address the challenges of dynamic, containerized deployments. In contrast, monolithic systems benefit from a broader array of established test generation methodologies.

In summary, the metrics introduced in [59] are crucial for evaluating the quality and comprehensiveness of microservices tests. These metrics, combined with non-functional testing metrics, provide a thorough assessment of the test coverage and effectiveness in microservices-based systems. However, the dynamic nature of microservices and the lack of dedicated test case generation tools present ongoing challenges in achieving comprehensive and reliable testing.

5.3. Enhancing Quality (RQ2.3)

Effective test generation strengthens microservice quality. It should address all interactions, fault points, and performance constraints. Table 16 summarizes seven key strategies and explains how each enhances reliability. We include references to maintain alignment with the original discussion.

Table 16. Key strategies for enhancing quality in microservices (RQ2.3).

ID	Strategy	Focus or Approach	Key References
S1	**Comprehensive Coverage**	Tests should span the entire system. Metrics such as Microservice Endpoint Coverage, Test Case Endpoint Coverage, and Complete Test Suite Endpoint Coverage ensure thorough validation.	[59]
S2	**Mock Services**	Mocks stabilize testing by simulating external services in controlled environments. This preserves interaction chains and allows consistent inputs and outputs. Papers highlight improved reliability through isolation during integration and regression tests.	[15,18,55]
S3	**Early Defect Detection**	Diverse test-case generation uncovers faults before production. Frameworks that combine static and dynamic analysis accelerate detection of subtle issues. This prevents cascades triggered by minor code changes.	[14]
S4	**Validating Inter-Service Interactions**	Microservices depend on well-defined contracts. Consumer-driven tests ensure that updates in one service do not disrupt upstream or downstream partners. This preserves communication integrity across versions.	[43,50]
S5	**Performance and Scalability Assessment**	Performance-focused tools generate tests that mimic various load scenarios. These tests reveal bottlenecks and verify capacity for high-traffic demands. Metrics from these assessments guide optimization.	[21,46]
S6	**Resilience Testing**	Fault injection introduces controlled errors, such as network delays or crashes. Effective strategies systematically orchestrate realistic failure scenarios and validate inter-service communications to ensure fault tolerance in production.	[30,41,51]
S7	**Automation and CI**	Automated frameworks plug into CI pipelines. Developers receive fast feedback on whether new code breaks existing functionality. Rapid iteration maintains consistent quality in frequent releases.	[20,25]

We also noted that many performance frameworks incorporate contract or mock-based validations. This practice ensures that services meet latency and throughput targets under diverse conditions. Resilience-focused test generation is enhanced by early detection of performance bottlenecks since these issues can worsen when faults occur. Improvements in one testing area often enhance overall system reliability.

5.4. Relationship: Test Coverage and the Efficiency of Test Generation (RQ2.4)

Understanding the relationship between test generation efficiency and coverage significantly enhances microservices' dependability and fault tolerance. Effective testing aims to balance thoroughness with resource efficiency to maximize coverage and minimize redundancy. According to [59], end-to-end test coverage metrics must comprehensively address all critical paths within the system under test (SUT) by exercising service invocations or alternative validation methods. Comprehensive coverage ensures that every potential combination of service interactions is evaluated under diverse operational conditions.

Mock services significantly influence both test generation efficiency and coverage. According to [15], employing mock services enhances testing efficiency by simulating real services and concentrating resources exclusively on relevant components. This focused approach prevents wasteful consumption of resources on irrelevant sections and enables

early detection of errors that might otherwise remain undetected until later stages. Even under strict time constraints, mock services improve the ability to identify defects early, promoting higher-quality outcomes.

Effective test generation improves regression testing efficiency by expanding coverage and reducing manual effort [18,20]. Automated methods rigorously test complex service interactions but face challenges due to microservices' independent deployment and scalability [5]. Varying test outcomes require robust and adaptable metrics to manage inconsistencies.

Optimizing test generation efficiency and coverage is essential for developing reliable and fault-tolerant microservice architectures. Employing mock services and rigorous testing strategies significantly enhances system dependability, supports early defect detection, and ensures thorough evaluation of complex service interactions despite inherent microservice deployment variability.

Answer of RQ2: Impact of Test Generation on Microservices Quality
Test generation for microservices (RQ2.1) depends on several critical factors. These include variability in service interactions, independence among services, and cloud-native deployment environments. Such characteristics demand adaptive and context-aware testing techniques. Robust quality metrics (RQ2.2), such as endpoint coverage and performance benchmarks, are essential to comprehensively validate both functional and non-functional requirements. Effective test generation methods (RQ2.3), including fault injection, automated mock services, and integration with continuous integration and continuous delivery, support early defect detection and improved system resilience. However, achieving a balance between thorough test coverage and available computational resources (RQ2.4) remains complex. Excessive coverage can result in resource exhaustion, whereas insufficient testing may leave critical failures undetected. Future research should focus on enhancing automated test generation tools to optimize coverage while minimizing resource demands, thereby ensuring both scalability and reliability.

6. Discussion

Our systematic mapping study synthesized the current research landscape of test generation for microservices, highlighted critical approaches, and identified gaps. Our findings emphasize a growing research focus on automated test generation methods specifically tailored for microservices architectures. Techniques such as automated regression test generation [18,20], consumer-driven contract tests [43,50], and mock-based testing frameworks [15] were prevalent. These approaches reflect efforts to systematically ensure functional correctness and integration consistency across independently evolving services.

Notably, dedicated tools explicitly designed for microservices test generation remain scarce. Although frameworks such as EvoMaster [37] and Jolie-based JoT [29] have emerged, they currently offer limited support for dynamically evolving interfaces and distributed scenarios typical of microservices. This gap signifies a pressing opportunity for future research to enhance the adaptability and scalability of automated test case generation techniques, as corroborated by related papers [38].

Our comparison with previous academic papers [43,44] underscores consistency in recognizing interface evolution and decentralized deployments as central challenges unique to microservices. However, while earlier papers broadly discussed testing strategies, our review explicitly emphasized the significance and limitations of automated test generation.

By narrowing this focus, we clearly highlight where tool innovation and methodological advancements are needed.

Additionally, our study revealed the limited exploration of standardized metrics for evaluating the effectiveness of generated tests. This absence restricted objective comparison and improvement across approaches. Future research should prioritize establishing common benchmarks, similar to efforts observed in other software testing domains [65], to systematically measure and enhance the quality of generated tests.

In conclusion, our study provided a current snapshot of the approaches and challenges in microservices test generation and their impact on system quality. The results not only synthesize existing knowledge but also highlight open issues and evolving trends. By comparing our findings with prior work, we see a clear trajectory of microservices testing research moving toward greater automation, early detection of issues, and holistic quality assurance spanning functional and non-functional concerns. We hope that the identified gaps and future directions will inspire further innovations in microservice-specific testing methodologies. Addressing these will be key to enabling reliable, scalable, and secure microservices systems, thereby supporting the continued success of microservices as a software architecture paradigm. Ultimately, advancing automated test generation and adopting adaptive testing strategies in practice will help ensure that quality will keep pace with the rapid evolution and deployment of microservices, strengthening confidence in modern cloud-based software systems.

7. Threats to Validity

Like any empirical research, our systematic mapping study faced several validity threats that warrant attention.

Internal Validity: In the context of this systematic mapping study, internal validity refers to the extent to which our chosen methodology accurately captures relevant publications without introducing significant biases. Selection bias was one major threat; although we conducted pilot searches and carefully designed inclusion/exclusion criteria, we might have missed some relevant papers despite our thorough search across various databases (SpringerLink, ACM Digital Library, IEEE Xplore, Scopus, and Web of Science) and established inclusion criteria. Our report follows the 2020 PRISMA Statement, which provides a comprehensive guideline for systematic literature reviews and meta-analyses [11]. This ensured that we had followed a proper standard process. Figure 2 shows that we adhered to the PRISMA flow along with the Checklist, which is publicly available on GitHub at https://github.com/miaoti/SMS-Test-Generation-for-Microservices (accessed on 26 March 2025). Data extraction bias was another concern specific to our methodology. Each paper's data points (see Section 2, Table 8) were extracted using a standardized form. Two authors independently reviewed any ambiguous cases, reducing the subjectivity inherent in mapping studies. Nevertheless, residual interpretation bias could have still affected how we classified and categorized individual contributions.

External Validity: This area concerns the generalizability of the findings. Our findings might not be fully generalizable to all microservices testing practices, especially those used exclusively within industry reports, white papers, or private documentation. By focusing on academic, peer-reviewed sources, we may have overlooked practical insights or non-English papers. However, we prioritized peer-reviewed papers to ensure a certain level of methodological rigor and credibility. Future research should incorporate more industrial or grey papers for a broader perspective on real-world challenges and solutions.

Construct Validity: In our case, a potential threat was that the research questions and objectives might have been misunderstood. To prevent this, we defined concepts clearly and maintained consistency during data extraction and analysis. Another threat was that

we had operationalized concepts. In defining our central concepts (e.g., "test generation" and "microservices architecture"), we followed established papers to minimize ambiguity. However, there was a risk that some nuanced testing practices or architectural variations were simplified during categorization. For instance, certain hybrid approaches might not have fit neatly into our predefined categories (see Section 3).

Conclusion Validity: The main threat to conclusion validity was bias in data analysis and interpretation, where different researchers could have had various interpretations of similar data. This was minimized by involving several reviewers who cross-checked the findings from their independent analyses against each other. Statistical conclusion validity, though less central in our predominantly qualitative mapping study, remained relevant for interpreting descriptive frequencies and trends (see Section 3). We relied on straightforward descriptive statistics (e.g., percentages and count distributions) to ensure transparency and the reproducibility of our findings, thus minimizing the risk of inaccurate or overstated inferences.

8. Conclusions

In this systematic mapping study, we examined the landscape of test generation for microservices architectures, focusing on existing approaches, tools, and challenges. Our analysis revealed a significant emphasis on automated methods tailored to the unique demands of microservices, including techniques such as automated regression test generation, consumer-driven contract testing, and mock-based frameworks. Despite these advancements, there remains a scarcity of dedicated tools explicitly designed to address the dynamic and distributed nature of microservices environments. Notable tools like EvoMaster and JoT have emerged but offer limited support for evolving interfaces and complex distributed scenarios. This gap shows the need for future research to enhance the adaptability and scalability of automated test generation techniques.

Our findings align with previous studies that highlighted interface evolution and decentralized deployments as central challenges in microservices testing. However, our focused review on automated test generation brings to light specific areas where tool innovation and methodological advancements are critically needed. Additionally, the lack of standardized metrics for evaluating the effectiveness of generated tests hampers objective comparison and improvement across different approaches. Establishing common benchmarks should be a priority in future research to systematically measure and enhance test quality.

To recapitulate, while significant progress has been made in microservices testing, substantial opportunities remain to develop specialized test generation tools and comprehensive evaluation metrics. Addressing these areas will advance testing practices and ensure higher quality and reliability in microservice-based systems.

Author Contributions: Conceptualization, T.M. and E.S.; Methodology, T.M. and E.S.; Validation, A.I.S. and E.S.; Formal analysis, T.M.; Investigation, T.M.; Writing—original draft, T.M.; Writing—review & editing, A.I.S. and E.S.; Visualization, T.M.; Supervision, E.S.; Project administration, E.S. All authors have read and agreed to the published version of the manuscript and are accountable for all aspects of the work to ensure that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Dragoni, N.; Lanese, I.; Larsen, S.; Mazzara, M.; Mustafin, R.; Safina, L. Microservices: Yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering*; Springer: Berlin/Heidelberg, Germany, 2017; pp. 195–216.
2. Fowler, M.; Lewis, J. Microservices: A Definition of This New Architectural Term. Available online: https://martinfowler.com/articles/microservices.html (accessed on 12 March 2025).
3. Newman, S. *Building Microservices: Designing Fine-Grained Systems*; O'Reilly Media, Inc.: Newton, MA, USA, 2015.
4. Christian, J.; Steven.; Kurniawan, A.; Anggreainy, M.S. Analyzing Microservices and Monolithic Systems: Key Factors in Architecture, Development, and Operations. In Proceedings of the 2023 6th International Conference of Computer and Informatics Engineering (IC2IE), Lombok, Indonesia, 14–15 September 2023; pp. 64–69. [CrossRef]
5. Eismann, S.; Bezemer, C.P.; Shang, W.; Okanović, D.; van Hoorn, A. Microservices: A Performance Tester's Dream or Nightmare? In Proceedings of the ACM/SPEC International Conference on Performance Engineering, New York, NY, USA, 25–30 April 2020; pp. 138–149. [CrossRef]
6. Panahandeh, M.; Miller, J. A Systematic Review on Microservice Testing. Preprint, Version 1. Research Square. 2023. Available online: https://www.researchsquare.com/article/rs-3158138/v1 (accessed on 17 July 2023). [CrossRef]
7. Petersen, K.; Vakkalanka, S.; Kuzniarz, L. Guidelines for conducting systematic mapping studies in software engineering: An update. *Inf. Softw. Technol.* **2015**, *64*, 1–18. [CrossRef]
8. Ahmed, B.S.; Bures, M.; Frajtak, K.; Cerny, T. Aspects of Quality in Internet of Things (IoT) Solutions: A Systematic Mapping Study. *IEEE Access* **2019**, *7*, 13758–13780. [CrossRef]
9. Dyba, T.; Dingsoyr, T.; Hanssen, G.K. Applying Systematic Reviews to Diverse Study Types: An Experience Report. In Proceedings of the First International Symposium on Empirical Software Engineering and Measurement (ESEM 2007), Madrid, Spain, 20–21 September 2007; pp. 225–234. [CrossRef]
10. Chadegani, A.A.; Salehi, H.; Yunus, M.M.; Farhadi, H.; Fooladi, M.; Farhadi, M.; Ebrahim, N.A. A Comparison between Two Main Academic Literature Collections: Web of Science and Scopus Databases. *arXiv* **2013**, arXiv:1305.0377. [CrossRef]
11. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [CrossRef] [PubMed]
12. Wohlin, C. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, New York, NY, USA, 13–14 May 2014.
13. Rahman, M.; Gao, J. A Reusable Automated Acceptance Testing Architecture for Microservices in Behavior-Driven Development. In Proceedings of the 2015 IEEE Symposium on Service-Oriented System Engineering, San Francisco, CA, USA, 30 March–3 April 2015; pp. 321–325. [CrossRef]
14. Giamattei, L.; Guerriero, A.; Pietrantuono, R.; Russo, S. Automated Grey-Box Testing of Microservice Architectures. In Proceedings of the 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), Guangzhou, China, 5–9 December 2022; pp. 640–650. [CrossRef]
15. Ashikhmin, N.; Radchenko, G.; Tchernykh, A. RAML-Based Mock Service Generator for Microservice Applications Testing. In *Supercomputing: Third Russian Supercomputing Days, RuSCDays 2017, Moscow, Russia, 25–26 September 2017*; Voevodin, V., Sobolev, S., Eds.; Springer International Publishing: Cham, Switzerland, 2017; pp. 456–467. [CrossRef]
16. de Camargo, A.; Salvadori, I.; Mello, R.d.S.; Siqueira, F. An architecture to automate performance tests on microservices. In *iiWAS '16: Proceedings of the 18th International Conference on Information Integration and Web-Based Applications and Services*; ACM Digital Library: New York, NY, USA, 2016; pp. 422–429. [CrossRef]
17. Ding, H.; Cheng, L.; Li, Q. An Automatic Test Data Generation Method for Microservice Application. In Proceedings of the 2020 International Conference on Computer Engineering and Application (ICCEA), Guangzhou, China, 18–20 March 2020; pp. 188–191. [CrossRef]
18. Gazzola, L.; Goldstein, M.; Mariani, L.; Segall, I.; Ussi, L. Automatic Ex-Vivo Regression Testing of Microservices. In Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test (AST 2020), Seoul, Republic of Korea, 25–26 May 2020; pp. 11–20. [CrossRef]
19. Janes, A.; Russo, B. Automatic Performance Monitoring and Regression Testing During the Transition from Monolith to Microservices. In Proceedings of the 2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Berlin, Germany, 28–31 October 2019; pp. 163–168. [CrossRef]

20. Kargar, M.J.; Hanifizade, A. Automation of Regression Test in Microservice Architecture. In Proceedings of the 2018 4th International Conference on Web Research (ICWR), Tehran, Iran, 25–26 April 2018; pp. 133–137. [CrossRef]
21. Smith, S.; Robinson, E.; Frederiksen, T.; Stevens, T.; Cerny, T.; Bures, M.; Taibi, D. Benchmarks for End-to-End Microservices Testing. In Proceedings of the 2023 IEEE International Conference on Service-Oriented System Engineering (SOSE), Athens, Greece, 17–20 July 2023; pp. 60–66. [CrossRef]
22. Elsner, D.; Bertagnolli, D.; Pretschner, A.; Klaus, R. Challenges in Regression Test Selection for End-to-End Testing of Microservice-Based Software Systems. In Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST 2022), Pittsburgh, PA, USA, 17–18 May 2022; pp. 1–5. [CrossRef]
23. Chen, L.; Wu, J.; Yang, H.; Zhang, K. A Microservice Regression Testing Selection Approach Based on Belief Propagation. *J. Cloud Comput.* **2023**, *12*, 21.
24. Duan, T.; Li, D.; Xuan, J.; Du, F.; Li, J.; Du, J.; Wu, S. Design and Implementation of Intelligent Automated Testing of Microservice Application. In Proceedings of the 2021 IEEE 5th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Xi'an, China, 15–17 October 2021; Volume 5, pp. 1306–1309. [CrossRef]
25. Wang, Y.; Cheng, L.; Sun, X. Design and Research of Microservice Application Automation Testing Framework. In Proceedings of the 2019 International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 20–22 December 2019; pp. 257–260. [CrossRef]
26. Lombardo, A.; Morabito, G.; Quattropani, S.; Ricci, C. Design, Implementation, and Testing of a Microservices-Based Digital Twins Framework for Network Management and Control. In Proceedings of the 2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM), Belfast, UK, 14–17 June 2022; pp. 590–595. [CrossRef]
27. Jiang, P.; Shen, Y.; Dai, Y. Efficient Software Test Management System Based on Microservice Architecture. In Proceedings of the 2022 IEEE 10th Joint International Information Technology and Artificial Intelligence Conference (ITAIC), Chongqing, China, 17–19 June 2022; Volume 10, pp. 2339–2343. [CrossRef]
28. Zhang, S.; Zhu, J.; Hao, B.; Sun, Y.; Nie, X.; Zhu, J.; Liu, X.; Li, X.; Ma, Y.; Pei, D. Fault Diagnosis for Test Alarms in Microservices through Multi-source Data. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024), Porto de Galinhas, Brazil, 15–19 July 2024; pp. 115–125. [CrossRef]
29. Giallorenzo, S.; Montesi, F.; Peressotti, M.; Rademacher, F.; Unwerawattana, N. JoT: A Jolie Framework for Testing Microservices. In Proceedings of the 25th IFIP WG 6.1 International Conference on Coordination Models and Languages (COORDINATION 2023), Lisbon, Portugal, 19–23 June 2023; Jongmans, S.S., Lopes, A., Eds.; Springer International Publishing: Cham, Switzerland, 2023; pp. 172–191. [CrossRef]
30. Meinke, K.; Nycander, P. Learning-Based Testing of Distributed Microservice Architectures: Correctness and Fault Injection. In Proceedings of the 13th International Conference on Software Engineering and Formal Methods (SEFM 2015), York, UK, 7–11 September 2015; Bianculli, D., Calinescu, R., Rumpe, B., Eds.; Springer: Berlin/Heidelberg, Germany, 2015; pp. 3–10. [CrossRef]
31. Camilli, M.; Guerriero, A.; Janes, A.; Russo, B.; Russo, S. Microservices Integrated Performance and Reliability Testing. In Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST 2022), Pittsburgh, PA, USA, 16–17 May 2022; pp. 29–39. [CrossRef]
32. Rattanukul, P.; Makaranond, C.; Watanakulcharus, P.; Ragkhitwetsagul, C.; Nearunchorn, T.; Visoottiviseth, V.; Choetkiertikul, M.; Sunetnanta, T. Microusity: A Testing Tool for Backends for Frontends (BFF) Microservice Systems. In Proceedings of the 31st IEEE/ACM International Conference on Program Comprehension (ICPC 2023), Melbourne, Australia, 15–16 May 2023; pp. 74–78. [CrossRef]
33. Wu, C.F.; Ma, S.P.; Shau, A.C.; Yeh, H.W. Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models. In Proceedings of the 29th Asia-Pacific Software Engineering Conference (APSEC 2022), Virtual Event, Japan, 6–9 December 2022; pp. 467–471. [CrossRef]
34. Ma, S.P.; Yang, Y.Y.; Lee, S.J.; Yeh, H.W. UTEMS: A Unit Testing Scheme for Event-driven Microservices. In Proceedings of the 10th International Conference on Dependable Systems and Their Applications (DSA 2023), Tokyo, Japan, 10–11 August 2023; pp. 591–592. [CrossRef]
35. Vassiliou-Gioles, T. Solving the Instance Identification Problem in Micro-service Testing. In Proceedings of the IFIP International Conference on Testing Software and Systems (ICTSS 2022), Almería, Spain, 27–29 September 2022; Clark, D., Menendez, H., Cavalli, A.R., Eds.; Springer International Publishing: Cham, Switzerland, 2022; pp. 189–195.
36. Christensen, H.B. Crunch: Automated Assessment of Microservice Architecture Assignments with Formative Feedback. In Proceedings of the 12th European Conference on Software Architecture (ECSA 2018), Madrid, Spain, 24–28 September 2018; Cuesta, C.E., Garlan, D., Pérez, J., Eds.; Springer International Publishing: Cham, Switzerland, 2018; Volume 11048, pp. 175–190. [CrossRef]
37. Zhang, M.; Arcuri, A.; Li, Y.; Liu, Y.; Xue, K. White-Box Fuzzing RPC-Based APIs with EvoMaster: An Industrial Case Study. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 122:1–122:38. [CrossRef]

38. Arcuri, A. RESTful API Automated Test Case Generation. In Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS), Prague, Czech Republic, 25–29 July 2017; pp. 9–20. [CrossRef]
39. Wang, S.; Mao, X.; Cao, Z.; Gao, Y.; Shen, Q.; Peng, C. NxtUnit: Automated Unit Test Generation for Go. In Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (EASE 2023), Oulu, Finland, 14–16 June 2023; pp. 176–179. [CrossRef]
40. Sahin, Ö.; Akay, B. A Discrete Dynamic Artificial Bee Colony with Hyper-Scout for RESTful Web Service API Test Suite Generation. *Appl. Soft Comput.* **2021**, *104*, 107246. [CrossRef]
41. Heorhiadi, V.; Rajagopalan, S.; Jamjoom, H.; Reiter, M.K.; Sekar, V. Gremlin: Systematic Resilience Testing of Microservices. In Proceedings of the 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS), Nara, Japan, 27–30 June 2016; pp. 57–66. [CrossRef]
42. Fischer, S.; Urbanke, P.; Ramler, R.; Steidl, M.; Felderer, M. An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study. In Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), New York, NY, USA, 2024; pp. 182–192. [CrossRef]
43. Simosa, M.; Siqueira, F. Contract Testing in Microservices-Based Systems: A Survey. In Proceedings of the 2023 IEEE 14th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 17–18 October 2023; pp. 312–317. [CrossRef]
44. Waseem, M.; Liang, P.; Márquez, G.; Salle, A.D. Testing Microservices Architecture-Based Applications: A Systematic Mapping Study. In Proceedings of the 27th Asia-Pacific Software Engineering Conference (APSEC 2020), Singapore, 1–4 December 2020; pp. 119–128. [CrossRef]
45. Zhang, J.; Jiang, S.; Wang, K.; Wang, R.; Liu, Q.; Yuan, X. Overview of Information System Testing Technology Under the "CLOUD + Microservices" Mode. In *Proceedings of the 5th International Conference on Computer and Communication Engineering (CCCE 2022), Beijing, China, 19–21 August 2022*; Neri, F., Du, K.L., Varadarajan, V.K., Angel-Antonio, S.B., Jiang, Z., Eds.; Springer International Publishing: Cham, Switzerland, 2022; pp. 62–74. [CrossRef]
46. Avritzer, A.; Ferme, V.; Janes, A.; Russo, B.; Schulz, H.; van Hoorn, A. A Quantitative Approach for the Assessment of Microservice Architecture Deployment Alternatives by Automated Performance Testing. In *Proceedings of the 12th European Conference on Software Architecture (ECSA 2018)*, Madrid, Spain, 24–28 September 2018; Cuesta, C.E., Garlan, D., Pérez, J., Eds.; Springer International Publishing: Cham, Switzerland, 2018; pp. 159–174. [CrossRef]
47. Gong, J.; Cai, L. Analysis for Microservice Architecture Application Quality Model and Testing Method. In Proceedings of the 2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD-Winter), Taichung, Taiwan, 6–8 December 2023; pp. 141–145. [CrossRef]
48. Giamattei, L.; Guerriero, A.; Pietrantuono, R.; Russo, S. Assessing Black-box Test Case Generation Techniques for Microservices. In *Proceedings of the Quality of Information and Communications Technology*, Talavera de la Reina, Spain, 12–14 September 2022; Vallecillo, A., Visser, J., Pérez-Castillo, R., Eds.; Springer International Publishing: Cham, Switezrland, pp. 46–60.
49. Sotomayor, J.P.; Allala, S.C.; Alt, P.; Phillips, J.; King, T.M.; Clarke, P.J. Comparison of Runtime Testing Tools for Microservices. In Proceedings of the 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), Milwaukee, WI, USA, 15–19 July 2019; Volume 2, pp. 356–361. [CrossRef]
50. Lehvä, J.; Mäkitalo, N.; Mikkonen, T. Consumer-Driven Contract Tests for Microservices: A Case Study. In Proceedings of the Product-Focused Software Process Improvement, Barcelona, Spain, 27—29 November 2019; Franch, X., Männistö, T., Martínez-Fernández, S., Eds.; Springer International Publishing: Cham, Switzerland, pp. 497–512.
51. Long, Z.; Wu, G.; Chen, X.; Cui, C.; Chen, W.; Wei, J. Fitness-guided Resilience Testing of Microservice-based Applications. In Proceedings of the 2020 IEEE International Conference on Web Services (ICWS), Beijing, China, 19–23 October 2020; pp. 151–158. [CrossRef]
52. De Angelis, E.; De Angelis, G.; Pellegrini, A.; Proietti, M. Inferring Relations Among Test Programs in Microservices Applications. In Proceedings of the 2021 IEEE International Conference on Service-Oriented System Engineering (SOSE), Oxford Brookes University, 23–26 August 2021; pp. 114–123. [CrossRef]
53. Schulz, H.; Angerstein, T.; Okanović, D.; van Hoorn, A. Microservice-Tailored Generation of Session-Based Workload Models for Representative Load Testing. In Proceedings of the 2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), Rennes, France, 22–24 October 2019; pp. 323–335. [CrossRef]
54. Lin, D.; Liping, F.; Jiajia, H.; Qingzhao, T.; Changhua, S.; Xiaohui, Z. Research on Microservice Application Testing Based on Mock Technology. In Proceedings of the 2020 International Conference on Virtual Reality and Intelligent Systems (ICVRIS), Zhangjiajie, China, 18–19 July 2020; pp. 815–819. [CrossRef]
55. Li, H.; Wang, J.; Dai, H.; Lv, B. Research on Microservice Application Testing System. In Proceedings of the 2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE), Dalian, China, 27–29 September 2020; pp. 363–368. [CrossRef]

56. Lu, Z. Research on Performance Optimization Method of Test Management System Based on Microservices. In Proceedings of the 2023 4th International Conference on Computer Engineering and Application (ICCEA), Hangzhou, China, 7–9 April 2023; pp. 445–451. [CrossRef]
57. Pei, Q.; Liang, Z.; Wang, Z.; Cui, L.; Long, Z.; Wu, G. Search-Based Performance Testing and Analysis for Microservice-Based Digital Power Applications. In Proceedings of the 2023 6th International Conference on Energy, Electrical and Power Engineering (CEEPE), Guangzhou, China, 12–14 May 2023; pp. 1522–1527. [CrossRef]
58. Wang, W.; Benea, A.; Ivancic, F. Zero-Config Fuzzing for Microservices. In Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), Echternach, Luxembourg, 11–15 September 2023; pp. 1840–1845. [CrossRef]
59. Abdelfattah, A.S.; Cerny, T.; Salazar, J.Y.; Lehman, A.; Hunter, J.; Bickham, A.; Taibi, D. End-to-End Test Coverage Metrics in Microservice Systems: An Automated Approach. In Proceedings of the Service-Oriented and Cloud Computing, Larnaca, Cyprus, 24–25 October 2023; Papadopoulos, G.A., Rademacher, F., Soldani, J., Eds.; Springer International Publishing: Cham, Switzerland, 2023; pp. 35–51.
60. Erl, T. *Service-Oriented Architecture: Concepts, Technology, and Design*; Prentice Hall PTR: Hoboken, NJ, USA, 2005; pp. 1–792.
61. Newman, S. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*; O'Reilly Media: Sebastopol, CA, USA, December 2020; pp. 1–270.
62. Auer, F.; Lenarduzzi, V.; Felderer, M.; Taibi, D. From monolithic systems to Microservices: An assessment framework. *Inf. Softw. Technol.* **2021**, *137*, 106600. [CrossRef]
63. Basiri, A.; Hochstein, L.; Jones, N.; Tucker, H. Automating chaos experiments in production. *arXiv* **2019**, arXiv:1905.04648.
64. JUnit Team. JUnit: A Programmer-Centric Testing Framework for Java. Available online: https://junit.org/junit5/ (accessed on 12 March 2024).
65. Rojas, J.M.; Fraser, G.; Arcuri, A. Seeding strategies in search-based unit test generation. *Softw. Test. Verif. Reliab.* **2016**, *26*, 366–401. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.